

RTU Mobile App

Setup & Development Guide

Complete guide for your development team to set up,
build, and extend the RTU prototype application

Flutter 3.x • Riverpod • GoRouter • Clean Architecture

Version 1.0 — March 2026

Table of Contents

- 1. Prerequisites & Environment Setup 3
- 2. Project Setup — Getting the App Running 4
- 3. Project Structure Walkthrough 6
- 4. Architecture Deep Dive 8
- 5. Theme System — How to Customize 10
- 6. Adding a New Feature Module 12
- 7. Mock Data — How It Works 14
- 8. Navigation — Adding New Routes 15
- 9. State Management with Riverpod 16
- 10. Development Workflow & Git Strategy 17
- 11. Sprint 2 — What to Build Next 19
- 12. Sprint 3 & 4 — Roadmap 21
- 13. Common Issues & Troubleshooting 22
- 14. Team Task Assignment 23
- 15. Checklist — Before RTU Presentation 24

1. Prerequisites & Environment Setup

Before anyone on the team touches the codebase, ensure these tools are installed on every development machine. This takes about 30 minutes for a fresh setup.

1.1 Required Software

Tool	Version	Installation	Why We Need It
Flutter SDK	3.19+ (latest stable)	flutter.dev/docs/get-started/install	Core framework — builds iOS + Android from single codebase
Dart SDK	Bundled with Flutter	Comes with Flutter installation	Programming language
Android Studio	Latest	developer.android.com/studio	Android emulator + SDK management
Xcode	15+ (macOS only)	Mac App Store	iOS simulator + build tools
VS Code	Latest	code.visualstudio.com	Primary code editor (or use Android Studio)
Git	2.30+	git-scm.com	Version control
Node.js	18+ LTS	nodejs.org	Mock API server (Sprint 2+)

1.2 VS Code Extensions (Recommended)

Extension	Publisher	Purpose
Flutter	Dart Code	Flutter development tools, hot reload, widget inspector
Dart	Dart Code	Dart language support, analysis, debugging
Riverpod Snippets	Robert Brunhage	Code snippets for Riverpod providers
Error Lens	Alexander	Inline error highlighting
GitLens	GitKraken	Git blame, history, and code authorship
Material Icon Theme	Philipp Kief	Better file icons in explorer

1.3 Verify Installation

Run this command to check everything is properly configured:

```
$ flutter doctor -v

# You should see checkmarks for:
# [✓] Flutter (Channel stable, 3.x.x)
# [✓] Android toolchain
# [✓] Xcode (macOS only)
# [✓] Chrome (for web debugging)
# [✓] Android Studio
# [✓] VS Code
```

■ If **flutter doctor** shows any [X] or [!] items, resolve them before proceeding. The most common issues are: Android SDK licenses not accepted (run **flutter doctor --android-licenses**), and Xcode command-line tools not installed (run **xcode-select --install**).

2. Project Setup — Getting the App Running

Follow these steps exactly in order. The entire process should take under 5 minutes once prerequisites are installed.

Step 1: Clone or Copy the Project

```
# If using Git (recommended):  
$ git clone <repo-url>  
$ cd rtu_app  
  
# Or if you have the folder directly:  
$ cd "RTU prototype/rtu_app"
```

Step 2: Install Dependencies

```
$ flutter pub get  
  
# This downloads all packages listed in pubspec.yaml  
# Expected output: "Got dependencies!"
```

Step 3: Run on a Device or Emulator

```
# List available devices:  
$ flutter devices  
  
# Run on default device:  
$ flutter run  
  
# Run on specific device:  
$ flutter run -d "iPhone 15 Pro"      # iOS Simulator  
$ flutter run -d emulator-5554        # Android Emulator  
$ flutter run -d chrome                # Web (for quick testing)
```

Step 4: Verify the App Launches

You should see the following flow:

- **Splash Screen** — Green background with animated RTU logo (2 seconds)
- **Login Screen** — Pre-filled with test credentials
- **Home Dashboard** — After tapping "Sign In", you land on the dashboard
- **Bottom Navigation** — 5 tabs: Home, Schedule, Academics, Services, Profile

✓ If you see the dashboard with the green header saying "Good Morning/Afternoon/Evening, J■nis!" — the setup is complete. You are ready to develop.

Step 5: Enable Hot Reload

While the app is running, use these shortcuts to speed up development:

Action	VS Code	Terminal	What It Does
Hot Reload	Ctrl+S (save)	r	Instantly reflects UI changes without losing state
Hot Restart	Ctrl+Shift+F5	R	Full restart — resets state but faster than cold start
Stop	Shift+F5	q	Stops the running app
Widget Inspector	Ctrl+Shift+P → "Flutter: Toggle Widget Inspector"	-	Visual debugging of widget tree

3. Project Structure Walkthrough

Understanding the folder structure is crucial before making any changes. The project follows **Clean Architecture** with a **feature-first** organization. Here is what each directory contains and when you would touch it:

3.1 Top-Level Structure

```
rtu_app/  
  ■■■ lib/                                # ALL Dart source code lives here  
  ■   ■■■ core/                          # Shared code used by ALL features  
  ■   ■■■ features/                      # Feature modules (each is self-contained)  
  ■   ■■■ navigation/                   # GoRouter config + shell screen  
  ■   ■■■ main.dart                     # App entry point  
  ■■■ assets/  
  ■   ■■■ images/                      # PNG/SVG images (RTU logo, illustrations)  
  ■   ■■■ icons/                       # Custom icon assets  
  ■   ■■■ mock_data/                   # JSON files with fake data  
  ■■■ test/                             # Unit and widget tests  
  ■■■ pubspec.yaml                     # Dependencies and asset declarations  
  ■■■ analysis_options.yaml            # Lint rules
```

3.2 Core Directory — Shared Foundation

```
lib/core/  
  ■■■ theme/  
  ■   ■■■ rtu_colors.dart                # Every color in the app (RTU brand + semantic)  
  ■   ■■■ rtu_typography.dart           # Every text style (display, headline, body, label)  
  ■   ■■■ rtu_spacing.dart              # Spacing scale + border radius tokens  
  ■   ■■■ rtu_theme.dart                 # Complete Material 3 ThemeData  
  ■   ■■■ theme.dart                    # Barrel export file  
  ■■■ constants/  
  ■   ■■■ route_constants.dart          # All route paths and names  
  ■■■ widgets/  
  ■   ■■■ rtu_card.dart                 # RTUCard, RTUAccentCard, RTUStatusCard  
  ■   ■■■ rtu_button.dart               # RTUPrimaryButton, RTUOutlinedButton, RTUChipButton  
  ■   ■■■ rtu_section_header.dart       # Section titles with "See All" action  
  ■   ■■■ rtu_shimmer.dart              # Loading placeholders  
  ■   ■■■ rtu_empty_state.dart          # Empty + Error states  
  ■   ■■■ widgets.dart                  # Barrel export  
  ■■■ network/                          # (Future) Dio HTTP client setup  
  ■■■ storage/                          # (Future) Hive/Isar local DB  
  ■■■ localization/                     # (Future) EN/LV translations
```

■ **Rule:** If a widget or utility is used by 2+ features, it belongs in **core/**. If it is specific to one feature, it stays inside that feature's directory.

3.3 Feature Module Structure

Every feature follows the same 3-layer pattern. Here is the schedule feature as an example:

```
lib/features/schedule/
  data/
  ■   data repositories/
  ■     data mock_schedule_repository.dart # Reads from JSON (swap for API later)
  domain/
  ■   domain entities/
  ■     ■   domain class_session.dart # Pure data class (no Flutter imports)
  ■     data repositories/
  ■       data schedule_repository.dart # Abstract interface (contract)
  presentation/
  ■   presentation providers/
  ■     presentation schedule_provider.dart # Riverpod state management
  ■   presentation screens/
  ■     presentation schedule_screen.dart # Main schedule page
  ■     presentation class_detail_screen.dart # Detail page
  ■   presentation widgets/
  ■     presentation day_selector.dart # Mon-Fri horizontal selector
  ■     presentation schedule_class_card.dart # Individual class card
```

■ **Critical Architecture Rule:** The **domain/** layer must NEVER import from **data/** or **presentation/**. Data flows upward: data → domain → presentation. This is what makes the mock-to-API swap painless.

4. Architecture Deep Dive

4.1 The Three Layers

Each feature module has exactly three layers. Understanding this is non-negotiable — it is the reason the prototype can be upgraded to production without rewriting screens.

Layer	Contains	Depends On	Who Changes It
Domain	Entities (data classes), Abstract Repository interfaces	Nothing — it is the innermost layer	Only when the data model changes
Data	Repository implementations, API clients, JSON parsing	Domain layer only	When switching from mock to real API
Presentation	Screens, Widgets, Riverpod providers	Domain layer only	When changing UI or adding screens

4.2 Mock-to-API Swap (The Key Selling Point)

Here is exactly how the transition from prototype to production works for each feature:

Current State (Prototype):

```
// schedule_provider.dart
final scheduleRepositoryProvider = Provider<ScheduleRepository>((ref) {
  return MockScheduleRepository(); // ← Reads from JSON files
});
```

Production State (After Getting API Access):

```
// schedule_provider.dart – ONLY this one line changes
final scheduleRepositoryProvider = Provider<ScheduleRepository>((ref) {
  final dio = ref.watch(dioProvider);
  return OrtusScheduleRepository(dio); // ← Calls real ORTUS API
});

// The schedule_screen.dart, day_selector.dart, class_card.dart
// ALL remain COMPLETELY UNTOUCHED. Zero UI changes needed.
```

✓ This is a **major selling point** when presenting to RTU. It shows our architecture is production-ready from day one — we are not throwing away prototype code, we are upgrading it.

4.3 State Management (Riverpod)

We use **Riverpod** (not Provider, not Bloc) for state management. Here is why and how:

- **FutureProvider** — For one-time data fetches (schedule, grades, events). Handles loading/error/data states automatically.
- **StateProvider** — For simple UI state (selected day, active tab, toggle values).
- **StateNotifierProvider** — For complex state with multiple actions (auth state, form state).
- **Provider** — For dependency injection (repository instances, services).

Pattern: Reading Data in a Widget

```
class MyScreen extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final scheduleAsync = ref.watch(weeklyScheduleProvider);

    return scheduleAsync.when(
      loading: () => CircularProgressIndicator(),
      error: (err, _) => RTUErrorState(onRetry: () => ref.invalidate(weeklyScheduleProvider)),
      data: (classes) => ListView.builder(
        itemCount: classes.length,
        itemBuilder: (ctx, i) => ClassCard(classes[i]),
      ),
    );
  }
}
```

5. Theme System — How to Customize

The RTU theme is centralized in **lib/core/theme/**. Every color, font size, and spacing value used anywhere in the app comes from these files. Never hardcode colors or sizes in screen widgets.

5.1 Using Colors

```
// ■ CORRECT – always use RTUColors
Container(color: RTUColors.primary)
Text("Hello", style: TextStyle(color: RTUColors.textSecondary))

// ■ WRONG – never hardcode hex values
Container(color: Color(0xFF005C3A))
Text("Hello", style: TextStyle(color: Colors.grey))
```

5.2 Using Typography

```
// ■ CORRECT – use RTUTypography
Text("Title", style: RTUTypography.headlineMedium)
Text("Body text", style: RTUTypography.bodyMedium)
Text("Caption", style: RTUTypography.labelSmall)

// Modify a style (e.g., change color):
Text("Green title", style: RTUTypography.titleLarge.copyWith(
  color: RTUColors.primary,
))
```

5.3 Using Spacing

```
// ■ CORRECT – use RTUSpacing constants
Padding(padding: EdgeInsets.all(RTUSpacing.lg))      // 16px
SizedBox(height: RTUSpacing.xxl)                    // 24px
BorderRadius.circular(RTUSpacing.radiusMd)          // 12px

// ■ WRONG – never use magic numbers
Padding(padding: EdgeInsets.all(16)) // Where did 16 come from?
```

5.4 If RTU Provides Their Brand Assets

When RTU shares their official brand guidelines or FF Mark Pro font files:

- Place font files in **assets/fonts/**
- Uncomment the **fonts:** section in **pubspec.yaml**
- In **rtu_typography.dart**, replace **GoogleFonts.inter(...)** with **TextStyle(fontFamily: "FFMarkPro", ...)**
- Update hex values in **rtu_colors.dart** if their official palette differs from our extracted values

6. Adding a New Feature Module

This is the step-by-step process every developer should follow when building a new feature (e.g., the Fees module). It ensures consistency across the codebase.

Step 1: Create the Folder Structure

```
$ mkdir -p lib/features/fees/data/repositories
$ mkdir -p lib/features/fees/data/models
$ mkdir -p lib/features/fees/domain/entities
$ mkdir -p lib/features/fees/domain/repositories
$ mkdir -p lib/features/fees/presentation/screens
$ mkdir -p lib/features/fees/presentation/providers
$ mkdir -p lib/features/fees/presentation/widgets
```

Step 2: Define the Entity (Domain Layer)

```
// lib/features/fees/domain/entities/invoice.dart
class Invoice extends Equatable {
  final String id;
  final String description;
  final double amount;
  final DateTime dueDate;
  final String status; // paid, pending, overdue
  // ...
}
```

Step 3: Define the Repository Interface (Domain Layer)

```
// lib/features/fees/domain/repositories/fees_repository.dart
abstract class FeesRepository {
  Future<List<Invoice>> getInvoices();
  Future<double> getOutstandingBalance();
}
```

Step 4: Create Mock Data

```
// assets/mock_data/fees.json
{ "invoices": [
  { "id": "inv-001", "description": "Tuition Spring 2026",
    "amount": 1250.00, "dueDate": "2026-04-05", "status": "pending" }
]
}
```

Step 5: Implement Mock Repository (Data Layer)

```
// lib/features/fees/data/repositories/mock_fees_repository.dart
class MockFeesRepository implements FeesRepository {
  @override
  Future<List<Invoice>> getInvoices() async {
    final json = await rootBundle.loadString("assets/mock_data/fees.json");
    // Parse and return...
  }
}
```

Step 6: Create Riverpod Provider

```
// lib/features/fees/presentation/providers/fees_provider.dart
final feesRepositoryProvider = Provider<FeesRepository>((ref) {
  return MockFeesRepository();
});

final invoicesProvider = FutureProvider<List<Invoice>>((ref) {
  return ref.watch(feesRepositoryProvider).getInvoices();
});
```

Step 7: Build the Screen

Use **ConsumerWidget**, **ref.watch()** the provider, and handle loading/error/data states with **.when()**.
Use shared widgets from **core/widgets/**.

Step 8: Register the Route

Add the route path to **route_constants.dart** and add a **GoRoute** entry in **app_router.dart**.

7. Mock Data — How It Works

All prototype data lives in **assets/mock_data/** as JSON files. The mock repositories read these files using **rootBundle.loadString()** and parse them into domain entities.

Current Mock Data Files

File	Content	Records
student.json	Student profile (Jānis Bērziņš, Computer Science, Semester 5)	1 student
schedule.json	Weekly timetable with 12 classes across Mon-Fri	12 class sessions
grades.json	Grades for 3 semesters with GPA history	3 semesters, 21 courses
events.json	University events (hackathons, career fair, lectures)	7 events

Adding New Mock Data

- Create a new JSON file in **assets/mock_data/**
- Use realistic RTU data (real building names, course codes, Latvian names)
- **Important:** Add the file to the **assets:** section in **pubspec.yaml** if you add a new directory
- The mock data path **assets/mock_data/** is already registered as an asset folder

■ Use authentic RTU data in mocks — real building names (Sēkļa ēka I, Sēkļa ēka II), real faculty names, real course code formats (DatZ3001, MatZ2001). This makes the demo feel real to RTU stakeholders.

8. Navigation — Adding New Routes

Navigation is managed by **GoRouter** with a **ShellRoute** that provides persistent bottom navigation across all main tabs.

Adding a New Screen (within a tab)

```
// 1. Add path constant in route_constants.dart:
static const String fees = '/services/fees';

// 2. Add GoRoute inside the services ShellRoute in app_router.dart:
GoRoute(
  path: 'fees',
  name: RouteNames.fees,
  parentNavigatorKey: _rootNavigatorKey, // Opens above bottom nav
  builder: (context, state) => const FeesScreen(),
),

// 3. Navigate to it:
context.push('/services/fees'); // Push (shows back button)
context.go('/services/fees');    // Go (replaces current route)
```

■ **push** vs **go**: Use **context.push()** for detail screens that need a back button. Use **context.go()** for tab-level navigation that replaces the current screen.

9. State Management with Riverpod

Every piece of state in the app goes through Riverpod. Here are the patterns to follow:

Use Case	Provider Type	Example
Fetch data once	FutureProvider	weeklyScheduleProvider — loads schedule JSON
Fetch with parameter	FutureProvider.family	dayScheduleProvider(dayOfWeek) — filter by day
Simple toggle/selection	StateProvider	selectedDayProvider — which day is selected
Complex state + actions	StateNotifierProvider	AuthNotifier — login/logout/token management
Dependency injection	Provider	scheduleRepositoryProvider — swap mock/real

Refresh Pattern

```
// Pull-to-refresh: invalidate the provider to re-fetch
RefreshIndicator(
  onRefresh: () async {
    ref.invalidate(weeklyScheduleProvider);
  },
  child: ...
)
```

10. Development Workflow & Git Strategy

10.1 Branch Strategy

```
main                # Production-ready code (protected)
■■■ develop         # Integration branch (all features merge here)
■■■ feature/schedule # Feature branches (one per feature)
■■■ feature/grades
■■■ feature/events
■■■ feature/fees
■■■ bugfix/xxx       # Bug fix branches
```

10.2 Daily Workflow

- **Morning:** Pull latest from **develop**, create or continue your feature branch
- **During day:** Commit frequently with clear messages. Use hot reload constantly.
- **End of day:** Push your branch, create/update PR if feature is reviewable
- **Code reviews:** Every PR needs approval from Parikshit before merging to develop

10.3 Commit Message Format

```
feat(schedule): add day selector with animated transitions
fix(grades): correct GPA calculation for pass/fail courses
style(theme): update primary green to match RTU brand guide
refactor(auth): extract login form into separate widget
docs: update setup guide with iOS simulator instructions
```

10.4 PR Checklist

- No hardcoded colors, font sizes, or spacing values
- Uses shared widgets from **core/widgets/** where applicable
- Follows the 3-layer architecture (domain/data/presentation)
- Loading and error states are handled
- No **print()** statements left in code
- Works on both iOS and Android (test on both if possible)

11. Sprint 2 — What to Build Next (Weeks 3-4)

Sprint 1 (foundation) is complete. Sprint 2 focuses on deepening the three most impactful features. Here is exactly what each developer should build:

Developer 1 — Home & Auth Polish

Task	Days	Details
Onboarding screens	1	Create 3-screen onboarding flow (swipeable) shown on first launch
Home dashboard refinement	1	Make stat cards dynamic (read from providers, not hardcoded)
Announcements data layer	1	Create mock repository + provider for announcements
Search functionality	2	Global search bar on Home screen — search courses, rooms, events
Pull-to-refresh everywhere	1	Add RefreshIndicator to all data-driven screens

Developer 2 — Schedule Deep Dive

Task	Days	Details
Week view grid	3	Full 5-day grid view using table_calendar package, color-coded blocks
Current class indicator	1	Highlight the currently active class with a pulsing border + countdown
Class detail push screen	1	Full-page detail with course info, professor contact, room on map
Schedule filters	1	Filter by course type (Lecture/Lab/Seminar) or by professor
Export to calendar	1	Generate iCal format + share sheet for adding to device calendar

Developer 3 — Grades & Analytics

Task	Days	Details
Grades data layer	1	Create GradesRepository, entity models, mock repository with provider
GPA trend chart	2	Line chart using fl_chart showing GPA across all semesters

Grade distribution pie chart	1	Pie chart showing grade breakdown (A/B/C/D/F percentages)
Semester detail screen	1	Tap semester → see all courses, grades, weighted GPA calculation
Course detail with assessments	1	Show breakdown: midterm 30%, final 50%, homework 20%
Unit tests	1	Test grade calculations, GPA computation, and repository parsing

12. Sprint 3 & 4 — Remaining Roadmap

Sprint 3 (Weeks 5-6): Extended Features

- **Events Calendar** — Full calendar view with month/week/list, event categories, RSVP, bookmarks
- **E-Learning Hub** — Course catalog, material browser, assignment tracker with status badges
- **Fees Module** — Invoice list, payment history, fee breakdown, overdue highlights
- **Admin Services** — Service catalog, document request forms, FAQ
- **Notification Center** — Full notification feed with deep linking to relevant screens
- **Library** — Catalog search, book availability, opening hours

Sprint 4 (Weeks 7-8): Polish & Demo

- **Campus Map** — Interactive map with building markers and room search
- **Language Switch** — Full EN/LV toggle with complete translations
- **Animations** — Page transitions, hero animations, micro-interactions
- **Performance** — Lazy loading, image caching, widget optimization
- **Edge Cases** — Empty states, error states, offline mode, long text handling
- **Demo Build** — TestFlight (iOS) + Firebase App Distribution (Android)
- **Presentation Materials** — Demo script, walkthrough video, architecture slides

13. Common Issues & Troubleshooting

Problem	Cause	Solution
flutter pub get fails	Dependency version conflict	Run flutter pub upgrade. If still failing, delete pubspec.lock and re-run.
Hot reload not working	Structural change (new file, new provider)	Use Hot Restart (R) instead. Hot reload only works for UI changes.
White/blank screen after login	Route not registered in GoRouter	Check that the route path exists in app_router.dart and screen is imported.
Asset not found error	File not in pubspec.yaml assets	Ensure the assets/ directory is listed under flutter: assets: in pubspec.yaml.
Text overflow on cards	Long Latvian text exceeds container	Add maxLines + overflow: TextOverflow.ellipsis to all Text widgets.
Colors look wrong	Hardcoded color instead of RTUColors	Search for Color(0x or Colors. in the file and replace with RTUColors.xxx.
"RenderFlex overflowed"	Column/Row content exceeds available space	Wrap content in SingleChildScrollView or use Expanded/Flexible widgets.
Provider not found	Missing ProviderScope in widget tree	Ensure ProviderScope wraps MaterialApp in main.dart (already set up).
iOS build fails	Pod install needed	Run: cd ios && pod install && cd .. then flutter run again.
Android build slow	First build downloads Gradle	First build takes 3-5 min. Subsequent builds are much faster.

14. Team Task Assignment

Here is the recommended task distribution. Parikshit reviews all PRs and handles RTU communication.

Developer	Sprint 2 Focus	Sprint 3 Focus	Sprint 4 Focus
Dev 1 (Frontend Lead)	Home polish, onboarding, search	Events calendar, admin services	Campus map, animations, demo build
Dev 2 (Feature Dev)	Schedule week view, filters, detail	E-Learning hub, library	Performance, edge cases, testing
Dev 3 (Data/Backend)	Grades repository, charts, tests	Fees module, notifications	Localization (EN/LV), CI/CD
Parikshit (Lead)	Architecture review, code reviews	Integration testing, RTU updates	Demo prep, presentation, final QA

Daily Standup Format (15 min)

- What I finished yesterday
- What I am working on today
- Any blockers?

15. Checklist — Before RTU Presentation

Go through this checklist in the final week before presenting to RTU:

App Quality

- ☐ Every screen loads without crashes on both iOS and Android
- ☐ No placeholder text visible ("Lorem ipsum", "TODO", "Coming soon")
- ☐ All mock data uses real RTU names, buildings, course codes
- ☐ Empty states are designed (not just blank screens)
- ☐ Error states show friendly messages with retry buttons
- ☐ Loading states use shimmer animations (not just spinners)
- ☐ RTU green (#005C3A) is consistent across all screens
- ☐ Text is readable — no tiny fonts, no truncated labels
- ☐ Bottom navigation works on every screen without bugs
- ☐ Transitions between screens are smooth (no jank)

Demo Devices

- ☐ iOS build installed on a physical iPhone via TestFlight
- ☐ Android build installed on a physical phone via Firebase App Distribution
- ☐ Both devices fully charged and in Do Not Disturb mode
- ☐ Backup demo device available
- ☐ Screen recording capability ready (in case of technical issues)

Presentation

- ☐ Demo script rehearsed (3-5 minute walkthrough)
- ☐ Key talking points: architecture diagram, mock-to-API swap, scalability
- ☐ Architecture slides prepared (use the Mermaid diagram)
- ☐ Answers prepared for likely questions (timeline, team size, cost, security)

☐ Development plan document (Word doc) printed or ready to share